

1. Observação importante

Em Java, o termo técnico mais usado para “atributo” é **field**, ou **campo**. Na prática didática de Orientação a Objetos, chamamos de **atributos** as variáveis declaradas dentro de uma classe para representar o estado de um objeto ou da própria classe. A documentação oficial da Oracle diferencia variáveis de instância, variáveis de classe, variáveis locais e parâmetros. Campos de instância são únicos para cada objeto; campos static pertencem à classe; variáveis locais ficam dentro de métodos ou blocos; parâmetros recebem valores enviados a métodos e construtores. ([Oracle Docs](#))

2. Tipos de atributos em Java

2.1 Atributo de instância

O atributo de instância é declarado dentro da classe, mas fora dos métodos, construtores ou blocos. Ele pertence a cada objeto criado a partir da classe. Isso significa que cada objeto possui sua própria cópia desse atributo.

Exemplo: em uma classe Aluno, cada aluno pode ter seu próprio nome, idade e matrícula. Esses dados variam de objeto para objeto.

```
public class Aluno {  
  
    String nome;  
  
    int idade;  
  
  
    public void exibirDados() {  
  
        System.out.println("Nome: " + nome);  
  
        System.out.println("Idade: " + idade);  
    }  
  
    public static void main(String[] args) {  
  
        Aluno aluno1 = new Aluno();  
  
        aluno1.nome = "Ana";  
  
        aluno1.idade = 16;  
  
  
        Aluno aluno2 = new Aluno();  
  
        aluno2.nome = "Carlos";  
  
        aluno2.idade = 17;  
  
  
        aluno1.exibirDados();  
  
        aluno2.exibirDados();  
    }  
}
```

```
}  
}
```

2.2 Atributo de classe ou atributo estático

O atributo de classe é declarado com a palavra-chave `static`. Ele pertence à classe, e não a cada objeto individualmente. Todos os objetos compartilham o mesmo valor desse atributo. A Oracle explica que campos `static` são associados à classe e existe uma única cópia compartilhada por todas as instâncias. (Oracle Docs)

```
public class ContadorAlunos {  
  
    String nome;  
  
    static int totalAlunos = 0;  
  
    public ContadorAlunos(String nome) {  
        this.nome = nome;  
        totalAlunos++;  
    }  
  
    public static void main(String[] args) {  
        ContadorAlunos a1 = new ContadorAlunos("Ana");  
        ContadorAlunos a2 = new ContadorAlunos("Pedro");  
  
        System.out.println("Total de alunos: " + ContadorAlunos.totalAlunos);  
    }  
}
```

2.3 Atributo constante com final

Um atributo declarado com final só pode receber valor uma vez. Depois de inicializado, não pode ser alterado. Segundo a especificação da linguagem Java, campos de classe e de instância podem ser declarados como final. ([Oracle Docs](#))

```
public class Produto {  
    final String codigo;  
  
    public Produto(String codigo) {  
        this.codigo = codigo;  
    }  
  
    public static void main(String[] args) {  
        Produto p = new Produto("P001");  
        System.out.println("Código: " + p.codigo);  
  
        // p.codigo = "P002"; // Erro: não pode alterar atributo final  
    }  
}
```

2.4 Constante de classe com static final

Quando usamos static final, criamos uma constante pertencente à classe. Esse tipo de atributo é muito usado para valores fixos, como taxa de juros, limite máximo, nome de sistema ou valor matemático.

Por convenção, constantes em Java são escritas com letras maiúsculas e palavras separadas por underline.

```
public class Matematica {  
  
    public static final double PI = 3.14159;  
  
  
    public static void main(String[] args) {  
  
        double raio = 5;  
  
        double area = PI * raio * raio;  
  
  
        System.out.println("Área do círculo: " + area);  
    }  
}
```

2.5 Atributo privado private

O atributo private só pode ser acessado dentro da própria classe. Ele é muito usado para aplicar **encapsulamento**, protegendo os dados do objeto contra acessos diretos indevidos.

Normalmente, atributos privados são acessados por métodos públicos chamados **getters** e **setters**.

```
public class ContaBancaria {  
  
    private double saldo;  
  
    public void depositar(double valor) {  
        if (valor > 0) {  
            saldo += valor;  
        }  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public static void main(String[] args) {  
        ContaBancaria conta = new ContaBancaria();  
        conta.depositar(100);  
  
        System.out.println("Saldo: " + conta.getSaldo());  
    }  
}
```

2.6 Atributo público public

O atributo public pode ser acessado diretamente por qualquer classe. Embora seja permitido, seu uso deve ser cuidadoso, pois pode quebrar o encapsulamento.

Em programação orientada a objetos, normalmente preferimos atributos private e métodos de acesso.

```
public class Pessoa {  
    public String nome;  
  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
        p.nome = "Mariana";  
  
        System.out.println("Nome: " + p.nome);  
    }  
}
```

2.7 Atributo protegido protected

O atributo protected pode ser acessado pela própria classe, por classes do mesmo pacote e por subclasses. Ele é comum em situações de herança.

```
class Animal {  
    protected String nome;  
}  
  
class Cachorro extends Animal {  
    public void latir() {  
        nome = "Rex";  
        System.out.println(nome + " está latindo.");  
    }  
}  
  
public class TesteAnimal {  
    public static void main(String[] args) {  
        Cachorro c = new Cachorro();  
        c.latir();  
    }  
}
```


2.8 Atributo com acesso padrão

Quando não usamos `public`, `private` ou `protected`, o atributo possui **acesso padrão**, também chamado de **package-private**. Ele pode ser acessado por classes do mesmo pacote.

```
class Curso {  
    String nomeCurso;  
  
    void exibirCurso() {  
        System.out.println("Curso: " + nomeCurso);  
    }  
}  
  
public class TesteCurso {  
    public static void main(String[] args) {  
        Curso c = new Curso();  
        c.nomeCurso = "Desenvolvimento de Sistemas";  
        c.exibirCurso();  
    }  
}
```

2.9 Atributo transient

O atributo transient é ignorado durante a serialização de objetos. Ele é usado quando não queremos salvar determinado dado em arquivo ou transmissão.

Exemplo comum: senha, token, dado temporário ou informação sensível.

```
import java.io.Serializable;

public class Usuario implements Serializable {

    String nome;

    transient String senha;

    public Usuario(String nome, String senha) {

        this.nome = nome;

        this.senha = senha;

    }

    public static void main(String[] args) {

        Usuario u = new Usuario("ana", "123456");

        System.out.println("Nome: " + u.nome);

        System.out.println("Senha: " + u.senha);

    }

}
```

2.10 Atributo volatile

O atributo volatile é usado em programação concorrente, quando várias threads podem acessar a mesma variável. Ele indica que o valor deve ser lido diretamente da memória principal, ajudando na visibilidade entre threads.

```
public class ControleThread {  
    private volatile boolean executando = true;  
  
    public void parar() {  
        executando = false;  
    }  
  
    public void executar() {  
        while (executando) {  
            System.out.println("Executando...");  
            break;  
        }  
    }  
  
    public static void main(String[] args) {  
        ControleThread controle = new ControleThread();  
        controle.executar();  
        controle.parar();  
    }  
}
```

2.11 Atributos de tipos primitivos

Atributos primitivos armazenam valores simples. Java possui oito tipos primitivos: byte, short, int, long, float, double, char e boolean. A linguagem Java é estaticamente tipada, ou seja, toda variável possui um tipo conhecido em tempo de compilação. ([Oracle Docs](#))

```
public class DadosAluno {  
  
    int idade = 16;  
  
    double media = 8.5;  
  
    char turma = 'A';  
  
    boolean aprovado = true;  
  
  
    public static void main(String[] args) {  
  
        DadosAluno aluno = new DadosAluno();  
  
  
        System.out.println(aluno.idade);  
  
        System.out.println(aluno.media);  
  
        System.out.println(aluno.turma);  
  
        System.out.println(aluno.aprovado);  
  
    }  
}
```

2.12 Atributos de referência

Atributos de referência armazenam referências para objetos, como String, arrays, objetos de outras classes, listas e coleções.

```
public class Endereco {  
    String cidade;  
  
    public Endereco(String cidade) {  
        this.cidade = cidade;  
    }  
}  
  
class AlunoComEndereco {  
    String nome;  
    Endereco endereco;  
  
    public AlunoComEndereco(String nome, Endereco endereco) {  
        this.nome = nome;  
        this.endereco = endereco;  
    }  
  
    public static void main(String[] args) {  
        Endereco e = new Endereco("São Paulo");  
        AlunoComEndereco aluno = new AlunoComEndereco("Lucas", e);  
  
        System.out.println(aluno.nome + " mora em " + aluno.endereco.cidade);  
    }  
}
```

2.13 Atributos do tipo array

Arrays em Java são objetos criados dinamicamente e podem ser atribuídos a variáveis do tipo Object, segundo a especificação da linguagem. ([Oracle Docs](#))

```
public class Boletim {  
  
    double[] notas = {8.0, 7.5, 9.0};  
  
    public void exibirNotas() {  
        for (double nota : notas) {  
            System.out.println("Nota: " + nota);  
        }  
    }  
  
    public static void main(String[] args) {  
        Boletim b = new Boletim();  
        b.exibirNotas();  
    }  
}
```

2.14 Atributos de enum

Um atributo do tipo enum armazena um conjunto fixo de valores possíveis. É útil quando queremos limitar as opções válidas.

```
enum SituacaoAluno {  
    APROVADO, RECUPERACAO, REPROVADO  
}  
  
public class Resultado {  
    SituacaoAluno situacao;  
  
    public Resultado(SituacaoAluno situacao) {  
        this.situacao = situacao;  
    }  
  
    public static void main(String[] args) {  
        Resultado r = new Resultado(SituacaoAluno.APROVADO);  
  
        System.out.println("Situação: " + r.situacao);  
    }  
}
```

3. Tabela-resumo dos tipos de atributos

Tipo de atributo	Palavra-chave	Característica principal
Instância	nenhuma obrigatória	Pertence a cada objeto
Classe/estático	static	Pertence à classe e é compartilhado
Constante	final	Só pode receber valor uma vez
Constante de classe	static final	Valor fixo compartilhado pela classe
Privado	private	Acesso apenas dentro da própria classe
Público	public	Acesso livre por outras classes
Protegido	protected	Acesso por herança e mesmo pacote
Padrão	nenhuma	Acesso dentro do mesmo pacote
Transiente	transient	Ignorado na serialização
Volátil	volatile	Usado em concorrência entre threads
Primitivo	int, double, etc.	Armazena valor simples
Referência	classe/objeto	Armazena referência para objeto
Array	[]	Armazena vários valores
Enum	enum	Limita valores possíveis

4. Exercícios

Atributo de instância

1. Crie uma classe Carro com atributos modelo e ano.
2. Crie dois objetos Carro com valores diferentes.
3. Explique por que cada objeto possui seu próprio valor.
4. Crie um método para exibir os dados do carro.
5. Altere o valor do atributo de apenas um objeto.

Atributo estático

1. Crie uma classe Produto com atributo estático totalProdutos.
2. Incremente esse atributo no construtor.
3. Crie três produtos e exiba o total.
4. Explique por que o valor é compartilhado.
5. Acesse o atributo usando o nome da classe.

Atributo final

1. Crie uma classe Documento com atributo final String numero.
2. Inicialize o atributo pelo construtor.
3. Tente alterar o valor e observe o erro.
4. Explique por que final protege o valor.
5. Crie outro objeto com outro número.

Atributo static final

1. Crie uma constante TAXA_JUROS.
2. Use a constante em um cálculo.
3. Escreva o nome da constante em letras maiúsculas.
4. Explique a diferença entre final e static final.
5. Crie uma classe Configuracao com uma constante NOME_SISTEMA.

Atributo private

1. Crie uma classe Aluno com atributo privado nota.
2. Crie métodos getNota() e setNota().
3. No setter, aceite apenas notas de 0 a 10.
4. Tente acessar o atributo diretamente.
5. Explique a relação com encapsulamento.

Atributo public

1. Crie uma classe Livro com atributo público titulo.
2. Acesse o atributo diretamente pelo objeto.
3. Altere o valor diretamente.
4. Explique uma desvantagem desse uso.
5. Reescreva usando private.

Atributo protected

1. Crie uma classe Funcionario com atributo protegido nome.
2. Crie uma subclasse Professor.
3. Acesse nome dentro da subclasse.
4. Crie um método para exibir o nome.
5. Explique quando usar protected.

Atributo padrão

1. Crie uma classe Disciplina com atributo sem modificador.
2. Acesse o atributo em outra classe do mesmo pacote.
3. Explique o conceito de package-private.
4. Compare com private.
5. Compare com public.

Atributo transient

1. Crie uma classe Login com usuario e transient senha.
2. Explique por que senha não deve ser serializada.
3. Crie um objeto Login.
4. Exiba os valores no console.
5. Pesquise onde a serialização é usada.

Atributo volatile

1. Crie um atributo booleano ativo.
2. Declare esse atributo como volatile.
3. Crie um método parar().
4. Explique a relação com threads.
5. Cite uma situação em que várias threads acessam o mesmo dado.

Atributos primitivos

1. Crie atributos int, double, char e boolean.
2. Exiba todos no console.
3. Explique a diferença entre int e double.
4. Explique a diferença entre char e String.
5. Crie uma média usando double.

Atributos de referência

1. Crie uma classe Endereco.
2. Crie uma classe Pessoa com atributo Endereco.
3. Instancie os dois objetos.
4. Exiba a cidade da pessoa.
5. Explique por que esse atributo é de referência.

Atributos array

1. Crie um atributo `int[]` `numeros`.
2. Inicialize o array com cinco valores.
3. Percorra usando `for`.
4. Calcule a soma dos valores.
5. Calcule a média dos valores.

Atributos enum

1. Crie um enum `StatusPedido`.
2. Use valores `ABERTO`, `PAGO`, `CANCELADO`.
3. Crie uma classe `Pedido` com atributo `StatusPedido`.
4. Exiba o status do pedido.
5. Explique por que enum evita valores inválidos.